

Angewandte KI &amp; KI-Engineering (AKE) · Block A – Vertiefung Deep Learning

## Lab Woche 1: MLP für MNIST

Aufgabenblatt · Basis-Lab (eigener Rechner)

Stefan Gnaase, PHWT · Wintersemester 2026/27 · Stand 28. September 2026

**Roter Faden.** Dieses Lab baut die Werkzeuge, mit denen im Rest des Semesters jedes Modell trainiert wird: Trainingsschleife, Optimierer, Regularisierung und die Fähigkeit, aus zwei Kurven die richtige Diagnose zu stellen. Das Trägermodell entsteht erst im nächsten Lab (*CNNs & Computer Vision*); die Repository-Struktur und die Trainingsschleife von heute werden dort unverändert weiterbenutzt.

### Organisatorisches – Lab Woche 1

|                 |                                                                                                                                                                                                                                 |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Termin          | KW 41 (08.10), 2-SWS-Lab-Block im Anschluss an den Theorieblock                                                                                                                                                                 |
| Skript          | <i>Vom Perzeptron zum Deep Network</i> , insbesondere Abschn. 3–8 (Theorie) und 9 (Lab)                                                                                                                                         |
| Material        | Kursrepository, Ordner <code>labs/woche01/</code> : <code>setup_check.ipynb</code> , <code>lab01_starter.ipynb</code> , <code>results_referenz/</code> (Referenz-Trainingslogs, falls eigene Läufe abbrechen)                   |
| Umgebung        | Python 3.11+, PyTorch (CPU-Build), torchvision, Jupyter, matplotlib – aus der versionierten Environment-Datei ( <code>uv sync</code> bzw. <code>conda env create -f environment.yml</code> )                                    |
| Daten           | MNIST vom Hochschul-Share (Ordner <code>data/MNIST</code> , 65 MB) nach <code>labs/woche01/data/</code> kopieren oder <code>AKE_DATA</code> auf den Share-Pfad setzen; Fallback: automatischer Download (12 MB)                 |
| Rechenbudget    | Referenzhardware 4 Kerne / 8 GB RAM, keine GPU: reine Rechenzeit $\approx$ 8–12 Min., davon $\approx$ 4 Min. Baseline, $\approx$ 3 Min. Optimierer-Sweep. Eine GPU oder Apple Silicon beschleunigt, ist aber nie Voraussetzung. |
| Erweitertes Lab | Cluster-Einstieg und 60-Läufe-Sweep (E1–E3, Skript Abschn. 9) sind optional und <i>nicht</i> Gegenstand dieses Blattes.                                                                                                         |
| Zeitplan        | 45 Min. Aufgabe 0 (Setup ist der kritische Pfad, nicht die Rechenzeit) · 45 Min. Aufgaben 1–2 · Rest und Aufgaben 3–4 als Selbststudium bis zur Abgabe                                                                          |

## Lernziele

Nach diesem Lab können Sie

- ein MLP in PyTorch definieren, trainieren und evaluieren, inklusive korrekter Trennung von Trainings-, Validierungs- und Testsplits (*Anwenden*);
- Backpropagation und Vanishing Gradients an Gl. (29) des Skripts erklären und das Verhalten tiefer Netze mit Sigmoid vs. ReLU vorhersagen (*Verstehen*);
- SGD, Momentum und Adam benennen und den Effekt der Lernrate aus einem Sweep ablesen (*Erinnern, Analysieren*);
- Trainingsverläufe nach Abb. 19 / Tab. 5 diagnostizieren und eine Regularisierungsstrategie mit Tab. 4 begründen (*Analysieren, Bewerten*).

## Aufgaben

### Aufgabe 0: Umgebung einrichten und prüfen (Pflicht, ohne Abgabe)

1. Kursrepository klonen, Umgebung aus der Environment-Datei anlegen. Der CPU-Build von PyTorch ist  $\approx$  200 MB; den CUDA-Build ( $\approx$  2,5 GB) nur installieren, wenn Sie eine NVIDIA-GPU haben.

2. MNIST-Ordner vom Hochschul-Share nach `labs/woche01/data/` kopieren.
3. `setup_check.ipynb` ausführen. Das Notebook prüft `torch.__version__`, Thread-Zahl, verfügbaren RAM, den MNIST-Bezug und misst die Laufzeit von 100 Trainingsschritten des Basis-MLP (Batch 64). Erwartung: hochgerechnet  $\leq 45$  s pro Epoche auf 4 Kernen.
4. Alle Prüfungen grün: weiter mit Aufgabe 1. Sonst: Notebook-Protokoll in der Sprechstunde vorzeigen. Typische Stolpersteine: Proxy (`pip config set global.proxy`), Python-Version  $< 3.11$ , fehlende Adminrechte (dann `uv` statt `conda`).

**Hausaufgabe bis Woche 6:** Docker Desktop (Windows: WSL2 und Virtualisierung im BIOS erforderlich) oder Podman Desktop installieren – wird im Lab *MLOps* gebraucht, Installationscheck folgt im Lab *Daten-Engineering*.

### Aufgabe 1: Baseline – MLP 784–128–128–10

Trainieren Sie das MLP aus Listing 1 des Skripts (zwei verdeckte Schichten à 128 Einheiten, ReLU, Logit-Ausgabe) mit Adam ( $\alpha = 10^{-3}$ ), Batch 64, 5 Epochen auf den 54 000 Trainingsbildern. Validierung auf 6 000 zurückgehaltenen Bildern, Testgenauigkeit *einmalig* am Ende.

1. Vervollständigen Sie die Klasse `MLP` im Starter-Notebook (variable Tiefe, Breite, Aktivierung, optional Dropout). Prüfen Sie die Parameterzahl: 118 282 (Bsp. 3.2).
2. Debug-Protokoll, Schritt 1 (Abschn. 8): Verlust beim ersten Batch vor dem Training ausgeben. Erwartung  $\approx \ln 10 = 2,30$ .
3. Trainieren Sie 5 Epochen und plotten Sie Trainings- und Validierungsverlust je Epoche. Erwartung: 97–98 % Testgenauigkeit.

**Interpretation:** Welchem der vier Grundmuster aus Abb. 19 entspricht Ihr Verlauf? Woran erkennen Sie, ob längeres Training noch etwas bringen würde? Warum darf die Ausgabeschicht kein Softmax enthalten (Abschn. 3.4)?

### Aufgabe 2: Architektur – Vanishing Gradients sichtbar machen

Bauen Sie ein 10-schichtiges Netz (10 verdeckte Schichten à 100 Einheiten), einmal mit Sigmoid und einmal mit ReLU, jeweils SGD mit  $\epsilon = 0,1$ .

1. Führen Sie *einen* Vorwärts- und Rückwärtsdurchlauf auf dem ersten Batch aus und messen Sie die Gradientennorm der ersten Schicht: `model.net[1].weight.grad.norm()`. Erweitern Sie die Messung auf alle Linear-Schichten und plotten Sie die Norm über der Schichtnummer (log-Skala).
2. Trainieren Sie beide Netze 3 Epochen auf dem 10 000er-Subset `d110k`. Zeichnen Sie  $\ln 10$  als Referenzlinie ein.

**Interpretation:** Erklären Sie das Verhältnis der Gradientennormen zwischen erster und letzter Schicht mit Gl. (29): Welcher Faktor in  $J_l = \text{diag}(\varphi'(\mathbf{a}_{l+1})) \mathbf{W}_{l+1}$  dämpft bei Sigmoid, welcher bei ReLU? Welche Zeile von Tab. 5 beschreibt den Sigmoid-Verlauf? Nennen Sie zwei Gegenmaßnahmen aus Abschn. 7.2 und ordnen Sie sie dem jeweiligen Faktor zu.

### Aufgabe 3: Optimierer $\times$ Lernrate

Für SGD, SGD+Momentum ( $\alpha = 0,9$ ) und Adam: Lernraten  $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$ , je 2 Epochen, Baseline-Architektur, gleicher Seed für alle 12 Läufe. **Rechenbudget:** Der Sweep läuft auf dem 10 000er-Subset (`d110k`), validiert wird auf dem vollen Validierungssplit ( $\approx 2$ –3 Min. auf 4 Kernen).

1. Protokollieren Sie den Validierungsverlust nach 2 Epochen als  $3 \times 4$ -Tabelle; markieren Sie Läufe mit `NaN/inf` oder Verlust  $\geq \ln 10$  als *divergiert*.

2. Plotten Sie die Validierungskurven (ein Diagramm je Optimierer, eine Linie je Lernrate, log-Skala).

**Interpretation:** Welche Kombination erreicht den geringsten Validierungsverlust, welche divergiert – und warum (Abb. 11, Gl. 18, Abb. 13)? Über wie viele Zehnerpotenzen der Lernrate ist jeder Optimierer „brauchbar“? Was folgt daraus für Schritt 3 des Debug-Protokolls?

#### Aufgabe 4: Regularisierungs-Ablation bei Datenknappheit

Reduzieren Sie das Training auf 1 000 Beispiele (d11k) und trainieren Sie die Baseline-Architektur 50 Epochen mit Adam ( $10^{-3}$ ) in vier Varianten: *keine Regularisierung* / *Weight Decay*  $10^{-3}$  / *Dropout* 0,5 / *Early Stopping* (patience 5, Parameterstand am Validierungsminimum behalten).

1. Plotten Sie Trainings- und Validierungsverlust aller Varianten in einem Diagramm, die Validierungsgenauigkeit in einem zweiten.
2. Tabelle: trainierte Epochen, bester Validierungsverlust, Testgenauigkeit (je Variante genau eine Testauswertung).
3. Optional: eine fünfte Variante Ihrer Wahl (z.B. Dropout + Early Stopping oder AdamW mit größerem  $\lambda$ ).

**Interpretation:** Beschreiben Sie die Schere der unregularisierten Variante (Abb. 14) – warum steigt der Validierungsverlust, während die Validierungsgenauigkeit kaum fällt (Tab. 5, letzte Zeile)? Warum wirkt Weight Decay  $10^{-3}$  bei Adam kaum (Abschn. 6.2)? Warum liegt bei Dropout der Validierungsverlust zeitweise unter dem Trainingsverlust? Welche Kombination empfehlen Sie – begründet mit der Reihenfolge aus Tab. 4?

#### Aufgabe 5: Optional – Backpropagation in NumPy

Implementieren Sie `backward_np` für das einschichtige MLP aus Bsp. 4.1 (ReLU, Softmax + Cross-Entropy, Mittelung über den Batch) und verifizieren Sie alle vier Gradienten per zentraler finiter Differenzen ( $\epsilon = 10^{-5}$ ) auf einem winzigen Netz in `float64`. Ziel: maximale Abweichung  $< 10^{-7}$ . Zählen Sie, wie viele Vorwärtsdurchläufe die numerische Prüfung benötigt, und vergleichen Sie mit Abschn. 4.3.

## Hinweise

### In der Praxis

**Debug-Protokoll vor jedem Experiment** (Skript Abschn. 8): (1) Verlust beim ersten Batch  $\approx \ln C$ ; (2) einen Minibatch überfitten – Verlust muss gegen 0 gehen, sonst ist es ein Codeproblem; (3) Lernrate über Zehnerpotenzen rastern, alles andere fest; (4) erst dann Architektur und Regularisierung – eine Variable zugleich. Loggen Sie jeden Lauf (Seed, Konfiguration, Kurven) – das Starter-Notebook schreibt `results/*.json`.

- **Testsplit-Regel:** Der Testsplit dient je Aufgabe genau einer finalen Bewertung. Wer ihn zur Auswahl von Hyperparametern nutzt, misst eine zu optimistische Generalisierung (Abschn. 6.1).
- **Reproduzierbarkeit:** `torch.manual_seed(0)` vor *jedem* Modell – sonst vergleichen Sie in Aufgabe 3 und 4 verschiedene Startpunkte statt verschiedener Hyperparameter.
- **`model.eval()` / `torch.no_grad()`:** in der Evaluationsschleife Pflicht; sonst bleibt Dropout aktiv und die Aufzeichnung des Graphen kostet Speicher (Abschn. 4.6, 6.4).
- **Fallback:** Bricht ein Lauf ab oder dauert er auf Ihrer Hardware zu lange, laden Sie das passende JSON aus `results_referenz/` und bearbeiten Sie die Interpretation damit. Kennzeichnen Sie das im Notebook.
- **Repository-Struktur:** Legen Sie jetzt `data/`, `src/`, `notebooks/`, `configs/`, `results/` an (Abschn. 9.3) – das Trägermodell aus dem nächsten Lab und alle MLOps-Labs bauen darauf auf.

**Organisatorisches – Abgabe Lab Woche 1**

|           |                                                                                                                                                                                                                                                                               |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Was       | Notebook <code>lab01_&lt;nachname&gt;.ipynb</code> ( <i>Restart &amp; Run All</i> lauffähig, mit Ausgaben) + Ordner <code>results/</code> mit den JSON-Logs; Aufgaben 1–4 vollständig, Aufgabe 5 optional                                                                     |
| Wann      | bis zum Beginn des Lab-Blocks in Woche 2 (KW 42), Upload im Kurs-Repository ( <code>abgaben/woche01/</code> )                                                                                                                                                                 |
| Bewertung | Übungsleistung mit Rückmeldung (Prüfungsleistung ist das Capstone-Projekt). Kriterien je Aufgabe: Experiment reproduzierbar und Kurven vorhanden · Interpretation mit korrektem Skript-Bezug (Gleichung, Tabelle, Abbildung) · Testsplit-Regel eingehalten. Aufgabe 5: Bonus. |
| Lösung    | Lösungs-Notebook wird nach der Abgabefrist im Repository freigeschaltet                                                                                                                                                                                                       |
| Fragen    | Sprechstunde im 2-SWS-Block; Setup-Probleme bitte mit Protokoll aus <code>setup_check.ipynb</code>                                                                                                                                                                            |